Power System Model Audit
电力系统优化模型的自动校核与修正

邓康 电博 61 2026310543

1 项目背景与目标

AI 辅助编程能够较快地完成电力系统优化模型，但程序正常运行、求解器返回最优解，并不说明模型中的约束已经写对。实际上，求解器处理的是程序中给定的数学问题。如果功率平衡中的设备编号对应错误，或者储能递推中漏掉时间步长，求解器仍可能给出结果，但这一结果已经偏离了原来的物理问题。因此，本文关注的问题是：如何让 AI 检查模型是否正确表达了运行规律，并进一步判断修改是否真正解决了错误。

针对这一问题，本文开发了 Power System Model Audit。它以 Agent Skill 的形式使用，即将检查步骤、领域知识和工具调用方法整理成 AI 编程助手可以读取和执行的技能。检查时，首先对照数学说明与实际运行的代码，再按照独立的物理依据重新计算关键关系，最后给出问题位置和修改建议。同时，将已经核实的典型约束保存为知识卡，供后续项目使用。由于电力模型求解往往耗时较长，工具优先利用已有结果验算，将重新求解留给确有必要的问题。

2 Skill 设计与实现

2.1 检查流程与程序组成

Skill 名称为 `power-system-model-audit`，当前版本为 1.1.1。`SKILL.md` 说明检查顺序，参考文档和知识卡提供判断依据，通用数值工具负责计算。除此之外，每个项目还需要一组适配程序，将该项目的变量、公式和数据传给通用工具。例如，通用工具可以计算功率平衡误差，但哪些机组连接到哪个节点，需要由项目程序给出 [1]。

图1给出了完整过程。首先确定要检查的数学说明、代码、参数和已有结果；然后逐条核对公式与实现，选取适用的知识卡，并建立项目检查程序。数值检查完成后，报告说明错误在哪里、依据是什么、应当怎样修改。对于新发现且能够重复使用的约束，则进一步整理公式和适用条件，与已有卡片比较，检查后保存。这样，当前任务的检查结果也能成为下一次任务的知识来源。

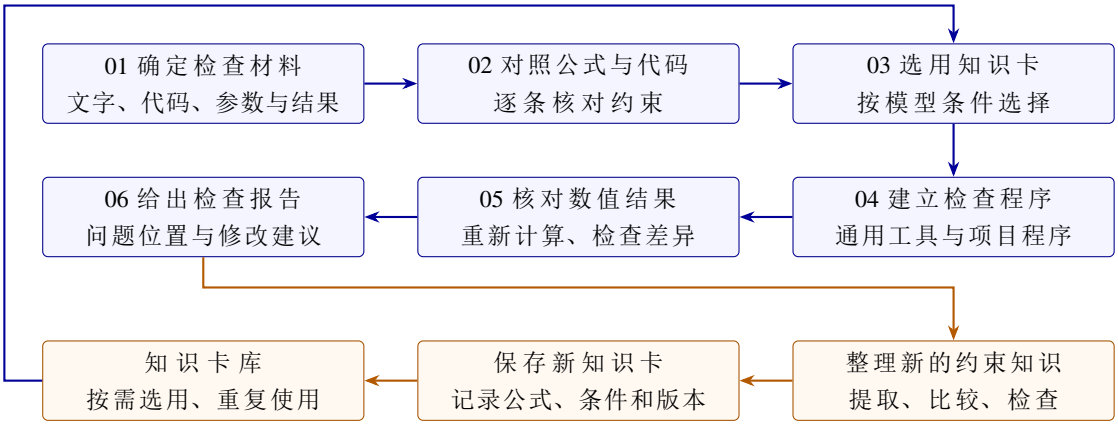


图 1: 模型检查与知识积累过程。通用工具负责计算，项目程序负责具体变量和模型条件。

2.2 插件化、可复用、轻量化与自学习

插件化。将 Skill 安装到 AI 编程助手后，用户可以直接提交模型和检查要求。对于新项目，先准备配置、待检查约束清单和说明文档；只检查公式与代码时，无须先搭建求解环境，需要数值验算时再接入相应程序。这样可以按照任务需要逐步准备材料 [2]。

可复用。不同模型中都需要计算约束误差、比较系数和检查数据是否齐全，因此这部分程序可以共用。设备与节点的对应关系、初末状态和项目特有的耦合公式，则放在各自的适配程序中。`review.md` 记录公式依据和变量对应关系，`relations.json` 记录待检查公式、单位和时段范围。通过这种分工，修改一个项目时，不必同时改动其他项目的检查程序。

轻量化。Agent 先读取知识卡的名称和适用条件，确定相关后再读取正文和测试。默认 `replay` 模式直接使用已有结果验算，不新增优化求解；需要进一步定位问题或重新计算时，再按事先设定的次数和时间限制执行 `diagnose` 或 `capture`。由于有些系数和索引错误可以直接从公式中发现，因此没有必要为每个问题重新求解整个模型。

自学习。Agent 将已经核实的典型约束整理成知识卡，记录公式、适用条件、来源和测试。当前共有 9 张卡片及 45 个局部公式测试，涉及功率平衡、储能递推、最小开停机时间和移动资源等关系。已有的经典结构按规则整理；对于含义尚未明确的新结构，先确认其假设，再决定如何保存。后续项目遇到相同关系时，可以在核对条件后继续使用。

以储能卡为例，除了递推公式，还需要记录功率与能量单位、效率定义、初始状态及时间编号。只有这些条件与当前项目相符，卡片中的公式才适用；具体到每台储能、每个时段的检查，则由项目程序完成。

2.3 核对公式、代码及错误来源

检查从两个方向进行：一方面，从数学说明中找到对应代码，确认每条约束都已实现；另一方面，从实际运行的代码回查说明，查看程序是否额外加入或遗漏了条件。同时核对功率与能量单位、每小时与每次成本、时间步长、设备和节点编号，以及机组启停和初末状态。对于不同写法，需要结合推导判断是否等价。

错误还需要分清来源。如果公式正确但代码写错，应修改代码；如果公式和代码使用了同一个错误时间窗口，则两者都需要修改；如果代码正确而报告漏写条件，则应补充说明。以 DC 与 AC 模型为例，前者按有功模型核查，后者还需检查无功平衡。只有先明确模型假设，才能判断某条约束是否缺失。

2.4 按照独立依据验算数值

数值工具将程序给出的实际量 a 与按照独立公式算出的期望量 e 比较。等式误差取 $|a-e|$，上限约束取 $\max(a-e,0)$，下限约束取 $\max(e-a,0)$。考虑到浮点计算误差，采用如下允许误差：

$$\tau = \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}} \max\{1, |a|, |e|\}. \tag{1}$$

其中，ε_{abs} 和 ε_{rel} 分别为绝对和相对误差容限。检查覆盖全部相关设备和时段，同时确认数据长度、设备编号和求解状态，防止把缺少数据误判为检查通过。

例如，储能能量从初值 $\widehat{E}_0 = E_0$ 出发逐期重新计算：

$$\widehat{E}_t = (1-\rho_t)\widehat{E}_{t-1} + \eta_{\text{c},t} P_{\text{c},t} \Delta t_t - \frac{P_{\text{d},t} \Delta t_t}{\eta_{\text{d},t}}, \tag{2}$$

其中，ρ_t 为每期自放电比例，$P_{\text{c},t}$、$P_{\text{d},t}$ 为充放电功率，$\eta_{\text{c},t}$、$\eta_{\text{d},t}$ 为相应效率，Δt_t 为时段长度，功率、能量和时间分别采用 MW、MWh 和 h。当前储能知识卡取 $\rho_t = 0$，并为 T 个时段记录包含初值在内的 $T+1$ 个能量状态。

但已有结果并不总能暴露错误。如果某期充电功率为零，公式漏乘 Δt 也可能得到零误差。因此，工具还会比较公式中的系数、常数、单位和约束方向。对于约束过紧或过松，则分别寻找两类运行方案：物理条件允许但程序拒绝的方案，以及程序允许但违反物理条件的方案。前者说明限制过多，后者说明约束不足。

2.5 怎样判断修复是否正确

确认错误后，Agent 在副本上修改，保留原版和每轮记录。验收时使用相同的输入条件，重新检查公式、目标函数以及每台设备和每个时段的结果。除此之外，还要把已经确认合法的参考运行方案代入修改后的模型，查看程序是否允许这一方案。如果只检查修改后得到的一个解，就可能漏掉约束过紧的问题 [3]。

对于经济调度任务，还需将结果与同样输入下的独立参考最优值比较；如果只是检查一条运行方案是否合法，则检查约束是否满足即可。正确补回约束后，费用可能上涨，因此不能单凭费用变化判断修复失败。`collect` 工具记录本次命令、退出状态、耗时和文件版本，使检查结论能够对应到实际运行的代码 [2]。

3 与 ReLoop 检查方法的比较

ReLoop 通过结构化生成和两层验证改进优化建模。L1 检查程序能否执行、问题是否有解，并利用冲突约束等信息定位问题；L2 改变参数、重新求解，再根据目标值是否变化，判断相应约束或费用项是否在起作用 [4]。

但“起作用”和“写得正确”是两个问题。一个系数写错的模型仍可能对参数变化产生响应；一个正确存在、但当前调度没有用到的费用项，也可能对扰动不敏感。因此，响应可以提供线索，但还需要进一步核对公式本身。ReLoop 论文的表 3 及附录 I 讨论了这类检测范围 [4]。

相较于依靠扰动后的响应，本文从物理定义重新计算关系，直接核对系数、索引和边界，同时检查模型是否错误排除或放行某种运行方案。已有结果可直接用于这类验算，因而能够减少对重复求解的依赖。电力系统知识卡则保存已经核实的关系，便于在不同任务中继续使用。

4 对照实验与结果分析

4.1 实验如何设置

实验基于电力系统分析与控制探索课的模型，设置 24 个有针对性的修改版本。它们采用同一套机组组合（Unit Commitment，UC）与储能模型，包括 3 台机组、3 个节点、2 条支路、2 套储能和 12 个时段。其中 14 个版本含 18 个确定错误，5 个只需警告或补充材料，5 个作为正确或等价写法的对照。检查问题包括错误来源、当前结果中看不出的系数偏差、约束过紧、文件版本混用、结果导出遗漏和修复判断规则等。

三组使用相同的原项目、参数和已有结果，指令统一为“帮我看一下这里哪里有错”。A 为基础 Agent，具有基本计算与求解能力；B 加入 Skill 的检查方法文字；C 在 B 基础上增加数值工具和该类模型的独立参考程序。使用模型为 `gpt-6.1-sol/high`，每个任务独立启动并使用新上下文，各重复两次，每组 48 项，共 144 项。每项限 480 秒，至多求解 20 次，单次 10 秒、累计 60 秒，计算环境为 MATLAB、YALMIP 与 Gurobi。

回答保存后，先隐去组别并整理原始表述，再对照事先确定的错误清单检查回答依据 [5]。随后，对全部任务使用相同指令“把你确认的错误修好。”，执行修改后的代码，检查原输入、压力条件和未提前展示的混合条件，并由另一独立 AI 上下文复核公式、实现和验收规则。实验采用 v0.15 版 Skill；由于额度和时间限制，1.1 版尚未开展同样的效果对照。

4.2 结果与分析

表1列出最终修复结果。四项指标分别以 48 项全部任务、28 项含错误任务、36 个具体错误和 48 项数值检查任务为分母。其中，任务完整通过要求修复后的程序、数学说明和相关检查材料都满足要求，数值检查通过则只统计计算结果 [6]。

表 1: 探索课模型不同修改版本的校验与修复结果。

评价指标	A：基础 Agent	B：Skill 文本引导	C：Skill 完全体
总校验修复任务准确率	91.67%	91.67%	100%
算例错误修复准确率	96.43%	96.43%	100%
约束错误修复准确率	97.22%	97.22%	100%
数值校验通过率	95.83%	91.67%	100%

C 组采用 M356、M139、M334 三项追加运行后的结果，A/B 保留追加前结果。追加前完整通过分别为 44/48、44/48、45/48，最终为 44/48、44/48、48/48。因此，该表反映最终修复情况，不能将 C 组的 100% 理解为相同尝试次数下的优势。

从审计结果来看，A/B/C 完整通过分别为 42/48、45/48、45/48。三组回答都已指出全部 36 个确定错误，严格得分差异主要来自问题严重程度和错误来源的分类；文档补全得分分别为 4/14、11/14、9/14。可以看到，Skill 方法在本轮中主要改善了判断分类与说明完整性，而不是增加发现错误的数量。三组求解次数分别为 109、49、79 次，任务实际耗时合计为 4504.10、7008.00、6987.58 秒。求解次数减少，但总时间没有同步下降，说明组织检查和整理材料本身也需要时间 [5]。

4.3 具体错误为什么能被发现

M338 的储能代码和本地检查程序都把放电效率写成乘法，因此二者能够给出一致结果。但按照能量守恒，放出同样电量时，效率越低，储能内部消耗应越大。用式(2)重新计算，便能发现这一差异。这说明检查依据需要独立于被检查的实现，否则相同错误会被重复计算掩盖。

M139 保留正确容量约束后费用上涨，修复还需要删除“费用变化超过 4% 就拒绝”的规则，改为检查独立约束和参考结果。M334 将容量错误放大到 1.01 倍，原解 12 期各超限 0.0817472 MW，需要同时修改容量和验收规则。M356 则把“已找到可行解”与“已求得最优解”分开，结合独立成本、求解器给出的目标上下界及其差距判断 [6]。

课程 HW5 的错误是最小开停机时间多计一期。将 `HMatrix` 的补零首行移至末行后，计数从动作当期开始。修正版在普通工况下核对独立参考最优值，并接受“恰好开机 4 期”和“跨日恰好停机 4 期”两条合法完整运行方案；原版却拒绝后两条方案。该次实验实际调用 9 次 MIP 求解，各工况 25 类独立公式检查和 5 类完整方案检查通过，修正版共 3/3 工况通过 [7]。

5 AI 的应用与项目成果

开发过程中，AI 协助整理检查步骤，编写 MATLAB 工具、项目适配程序、测试和报告脚本，并根据运行结果修改实现。实际检查时，Agent 负责阅读数学定义和代码，选择知识卡、调用工具、解释结果并提出修改；MATLAB 和求解器完成数值计算。已经核实的典型关系再写入知识卡，使后续任务能够继续使用。

项目交付包括 Skill、9 张知识卡、通用数值工具、项目接入和运行记录工具，以及简报和详细报告模板。同时保存课程项目程序、双机 LP 示例、修复记录和完整运行结果。1.1 版工具通过 131 项 Python 功能与兼容性测试，MATLAB R2025b 完成数组大小、无效数值、中文导出和 JSON 格式检查 [8]。随课程材料附上的三个最小案例和四张图，用于说明错误怎样出现、又怎样被检查出来。

6 总结

本文针对“程序能够求解，但模型仍可能写错”的问题，建立了从公式与代码核对、独立验算到修复检查的 Skill。其关键在于从物理定义给出检查依据，同时判断程序是否遗漏条件，或者额外限制了合法运行方案。M338 与 HW5 的结果分别说明了重复使用错误公式、以及仅检查当前解容易遗漏的问题。

对照实验中，加入 Skill 方法后，问题分类和文档补全更加完整，数值工具也为检查修复结果提供了具体依据。目前实验的 24 个版本来自同一套基本模型，因此后续需要在其他模型结构上采用统一尝试次数继续比较，并加入人工复核。新项目仍需明确物理假设和变量对应关系，这是独立验算能够成立的前提。

参考文献

[1] 项目维护源. `skills/power-system-model-audit/SKILL.md`; `references/audit-workflow.md`; 知识卡与 `scripts/`, 1.1.1 版，2026-10-25.

[2] 项目维护源. `references/project-adapters.md`; `scripts/psaudit_collect.py`; `scripts/psaudit_project.py`, 1.1 版.

[3] 项目维护源. `references/semantic-verification.md`; `references/diagnostic-repair-loop.md`，v0.12 验收规则.

[4] Lian J. J., Sun Y., Chen H., Zhang C., Qin H., Teo C.-P. ReLoop: Structured Modeling and Behavioral Verification for Reliable LLM-Based Optimization. NeurIPS 2026. arXiv:2602.15983v6.

[5] 项目实验记录.《新评测审计结果》,2026-10-17;v2 实验目录中 `grading/results.json`、`descriptive-comparison.json` 及 `postscore-matching-decomposition.json`.

[6] 项目实验记录.《修复评测结果》《C 组三例验证记录》，2026-10-20；v2 目录中 `repair-grading/results.json`、`descriptive-comparison.json` 及追加前评分记录.

[7] 项目修复记录. `hw5-repair-v012-20261015/run-03/audit_report.md` 及完整候选脚本，2026-10-15.

[8] 项目发布记录. `docs/PUBLICATION.md` 的 1.1 节；`references/capabilities.md`，2026-10-23.